

Analytical Policy Functions for Income Fluctuation Problems When the Value Function is Concave and Piecewise Linear*

Yuichiro Waki^{†‡}

Aoyama Gakuin University and the University of Queensland

October 4, 2026

Abstract

This paper studies fitted value iteration for discrete-time income fluctuation problems with a single asset. I show that, when the continuation value is represented by a concave piecewise-linear function, the consumption and saving policies can be obtained analytically. The analytical solution substantially accelerates the value function iteration algorithm with piecewise-linear interpolation, because it is inexpensive both to compute and to evaluate and eliminates the need for numerical optimization, while retaining its convergence property. It also lowers the cost of using a fine grid. Further acceleration is possible through policy function iteration, because calculating the value of a given policy function amounts to solving a sparse linear system of equations, where no additional computation is required to obtain nonzero elements of a sparse coefficient matrix. In numerical experiments, value iteration has computation time comparable to the standard endogenous grid method, and policy function iteration is faster and performs comparably to the continuous-time implicit method. Although piecewise-linear interpolation can be inaccurate in regions of high curvature, a simple modification improves accuracy by reducing the Euler equation error to the level of the standard endogenous grid method.

Keywords: Value function iteration; fitted value iteration; Piecewise-linear interpolation; Endogenous grid method; Income fluctuation problem

*This paper was previously circulated as “A Fast and Convergent Hybrid of Value Function Iteration and the Endogenous Grid Method for Discrete-Time Income Fluctuation Problems.” I am especially grateful to Masao Fukui and Akira Toda for their insightful comments. I also thank Toni Braun and Karen Kopecky for helpful suggestions, and participants at Meiji University, Hitotsubashi University, the 2025 Workshop of the Australasian Macroeconomics Society, and OzMac 2025 workshop for their comments. I am grateful for financial support from Japan Society for the Promotion of Science (JSPS) KAKENHI Grant-in-Aid for Scientific Research (A) Grant Number 22H00058.

[†]Associate Professor, Department of Economics, Aoyama Gakuin University. Email: ywaki@econ@gmail.com. Address: 4-4-25 Shibuya, Shibuya-ku, Tokyo, Japan. ORCID ID: 0000-0003-1929-3520

[‡]Honorary Senior Fellow, the School of Economics, the University of Queensland, Australia.

1 Introduction

Fitted value iteration algorithms are widely used to numerically solve continuous-state dynamic programming problems. These algorithms discretize the state space, update a value function estimate on a finite set of grid points at each step, and use these data to construct an approximate function over the entire state space via interpolation, regression, or other approximation methods. Various such methods have been proposed and their pros and cons have been extensively studied in the literature (see, e.g., [Judd \(1998\)](#); [Cai and Judd \(2014\)](#)).

Among these, this paper focuses on fitted value iteration with piecewise-linear (affine) interpolation. This method has several desirable properties. First, piecewise-linear interpolation preserves monotonicity and concavity, making it a useful approximation technique when the true value function is known to be both monotonic and concave. Second, it is a non-expansive approximation method; hence, fitted value iteration with piecewise-linear interpolation — provided that each optimization step is solved accurately — is guaranteed to converge to a unique numerical fixed point because the numerical Bellman operator is a contraction mapping ([Stachurski, 2008](#)).¹ This stability contrasts with that of some polynomial and spline approximation schemes, which need not preserve contraction and may fail to converge. Despite these desirable properties, piecewise-linear interpolation is often viewed as computationally unattractive: it produces non-smooth maximization problems through kinks in the objective function and may require many grid points for accuracy ([Judd, 1998](#); [Cai and Judd, 2014](#)). This trade-off has motivated work on smooth, shape-preserving approximation methods, including [Judd and Solnick \(1994\)](#) and [Cai and Judd \(2012\)](#).

This paper shows that the principal computational drawback of piecewise-linear fitted value iteration disappears in an important class of discrete-time income fluctuation (or consumption-saving) problems with one asset. In the problem, an agent maximizes the current utility from consumption plus the discounted continuation value as a function of next-period saving, subject to budget and borrowing constraints. The main result is that, if the continuation value is a concave, piecewise-linear (affine) function of saving, then the policy functions, i.e., the optimal consumption and saving choices as functions of current cash-on-hand, can be obtained analytically and are piecewise-linear in current cash-on-hand, with kinks located on an endogenous grid. This result is useful if the problem is such that the true value function is concave and if piecewise-linear interpolation is used to approximate the value function. The analytical solution is obtained by applying the idea of the standard endogenous grid method (EGM) proposed by [Carroll \(2006\)](#) to a piecewise-linear value function, exploiting its constant slope on each linear segment.

¹The convergence result is also shown elsewhere in the literature. See, e.g., [Santos and Vigo-Aguiar \(1998\)](#) and [Judd and Solnick \(1994\)](#).

The analytical policy functions substantially accelerate a fitted value function iteration (VFI) algorithm with piecewise-linear interpolation, alleviating two important computational drawbacks. First, the analytical policy functions eliminate the need for numerical optimization. Second, piecewise-linear interpolation often requires a fine grid to achieve a given accuracy when the curvature is large. The analytical policy functions are inexpensive to evaluate, allowing us to use a fine grid without much computational cost. Hence, the analytical policy functions improve the practical usefulness of VFI with piecewise-linear interpolation.

An important precursor to the present paper is [Barillas and Fernandez-Villaverde \(2007\)](#). They also combine the standard EGM with VFI to demonstrate a large efficiency gain relative to the standard VFI. There are two important differences between the present paper and theirs. First, in each optimization step, they use the standard EGM to obtain an approximate policy function on an endogenous grid from a derivative estimate of the continuation value function on a finite grid, and use interpolation (and extrapolation if necessary) for evaluation on the exogenous grid. The analytical policy function in the present paper solves the optimization problem exactly conditional on the piecewise-linear continuation value, whereas their policy function generally does not. Second, they do not consider acceleration through policy function iteration, whereas the present paper utilizes the PFI algorithm in [Rendahl \(2022\)](#) as I describe below.

The analytical policy functions have a simple structure. They are piecewise linear on an endogenous grid on the space of cash-on-hand, and the grid points are easy to compute. On each subinterval of cash-on-hand where they are linear, either consumption or saving is constant at an easily computable value. Hence, when evaluating the policy functions at a given value of cash-on-hand, first find a subinterval that contains it, then obtain either consumption or savings that is constant on that subinterval, and the other variable can be recovered from the budget constraint. There is no need for interpolation when evaluating the policy functions at off-grid points, which is particularly desirable when repeatedly evaluating them at many distinct values.

Further acceleration is possible by using the acceleration method proposed in [Rendahl \(2022\)](#), which is based on the policy function iteration (PFI) algorithm (Howard’s improvement algorithm) or the modified policy function iteration (MPFI) algorithm ([Puterman and Shin, 1978](#)). The main insight of [Rendahl \(2022\)](#) is that, once the policy function for savings is approximated with a discrete, sparse probability transition matrix on the savings grid, the value of following the approximated policy can be computed instantly. This is because the policy function iteration algorithm reduces to sparse linear algebra. The present paper complements Rendahl’s PFI algorithm in four respects. Given that piecewise-linear interpolation is used to represent a value function, (1) the analytical solution speeds up the policy-improvement step, and (2) the sparse probability matrix can be constructed directly from interpolation weights already computed during

policy construction. In addition, (3) the policy evaluation step, in theory, obtains exactly the value of following the analytical policy function forever, conditional on the piecewise-linear representation. Finally, (4) a concavification step is introduced to preserve the conditions needed for the next policy-improvement step.

I conduct some numerical exercises to evaluate the computation time and accuracy of VFI and PFI/MPFI algorithms by comparing them with two algorithms that are known to be fast: the standard EGM and the continuous-time, implicit method.² In terms of the computation time, VFI is similar to EGM in the benchmark, while PFI and MPFI are faster than EGM and have computation time comparable to the continuous-time method. Both PFI and MPFI are much more efficient than VFI, reducing computation time by 60% or more in the present paper’s numerical experiments. In terms of accuracy, a simple modification of the algorithm substantially improves accuracy. Piecewise-linear interpolation for the value function can be inaccurate in regions with high curvature. However, the Euler equation error can be significantly reduced by applying two or three steps of the standard EGM after computing the policy function using PFI. This modification reduces the Euler equation error to a level comparable to that of the standard EGM.

There are some limitations in the present paper’s algorithm. First, because it relies on the first-order condition to solve the optimization problem, the optimality of the analytical policy function is not guaranteed in non-concave problems. Second, even for concave problems, the analytical policy function in the present paper is only available when the endogenous state variable is one-dimensional. For concave problems with multidimensional state variables, the lower approximation method of [Fukushima and Waki \(2013\)](#) remains useful, yet the policy-improvement step requires numerical optimization. Extending the analytical result to multidimensional settings is left for future research.

This paper is organized as follows. Section 2 sets up a two-period consumption-saving problem and proves a key proposition regarding properties of the policy function. Section 3 describes the VFI algorithm using a deterministic neoclassical growth model. Section 4 describes the PFI algorithm, and Section 5 compares its performance with the standard EGM and with a continuous-time approach. In Section 6, I compare the Euler equation errors produced by the proposed method and the standard EGM, and introduce a simple modification that reduces the error. Section 7 concludes.

²For all numerical experiments, a MATLAB replication package is available at https://ywaki-econ.github.io/files/programs_VFIPWL.zip.

2 Main Theoretical Result

Consider a piecewise-linear function on a compact interval $\mathcal{K} = [k_{min}, k_{max}] \subset \mathbb{R}_+$. It is characterized by a pair consisting of a finite, increasing grid $\mathcal{K}_{grid} = \{k_1, k_2, \dots, k_{I_k}\}$ with $k_1 = k_{min}$ and $k_{I_k} = k_{max}$, and the function's values on the grid, $\{v_1, v_2, \dots, v_{I_k}\}$. Henceforth, I will take as exogenously given the grid $\mathcal{K}_{grid}$, and the function can be summarized by a finite vector, $\mathbf{v} = (v_1, v_2, \dots, v_{I_k})^\top$, which is a column vector. Let $\hat{V}(\cdot; \mathbf{v})$ denote the piecewise-linear function on $\mathcal{K}$ given $\mathbf{v}$ such that

$$\hat{V}(k; \mathbf{v}) = \frac{k_{i+1} - k}{k_{i+1} - k_i} v_i + \frac{k - k_i}{k_{i+1} - k_i} v_{i+1} \quad \text{if } k \in [k_i, k_{i+1}] \text{ for some } i.$$

Below I focus on a concave piecewise-linear function, and thus assume that $(v_{i+1} - v_i)/(k_{i+1} - k_i)$ is non-increasing in i .

Consider the following consumption-saving problem:

$$m(y; \mathbf{v}) := \max_{c, k'} \{u(c) + \beta \hat{V}(k'; \mathbf{v})\}$$

subject to the budget constraint

$$c + k' \leq y,$$

and the constraint on next-period capital, $k' \in \mathcal{K}$. Here, y denotes the cash-on-hand and k' denotes savings. The parameter $\beta \in (0, 1)$ is the preference discount factor and $\hat{V}$ represents the value from savings. The function u is a strictly increasing, strictly concave C^1 function satisfying Inada condition. It is also assumed that u' has an analytical inverse, e.g., $u(c) = \ln c$ or $u(c) = c^{1-\sigma}/(1-\sigma)$ for $\sigma > 0$ with $\sigma \neq 1$.

The above problem, which I denote by $\mathcal{P}(y; \mathbf{v})$, is indexed with cash-on-hand, y , and a value vector, $\mathbf{v}$. Let $\mathcal{Y}$ be the set of y 's such that the constraint set is non-empty. If the domain of u is $(0, \infty)$, then $\mathcal{Y} = \{y \in \mathbb{R}_+ | y > k_{min}\}$. If u is well-defined at $c = 0$, then $\mathcal{Y} = \{y \in \mathbb{R}_+ | y \geq k_{min}\}$.

Of interest are a solution and the maximized value, $m(y; \mathbf{v})$, to the problem $\mathcal{P}(y; \mathbf{v})$ for $y \in \mathcal{Y}$. The solution to the problem $\mathcal{P}(y; \mathbf{v})$ is unique for each $y \in \mathcal{Y}$, because u is strictly concave and the constraint set is convex. Viewed as a function of y , the solution is then the *policy functions* for consumption and saving, which I denote by $g_c(y; \mathbf{v})$ and $g_{k'}(y; \mathbf{v})$, respectively. Although the problem features kinks in the objective function and bound constraints, its solution and value are fully characterized by the following proposition.

Proposition 1 *Suppose $\mathbf{v}$ is such that $\hat{V}(\cdot; \mathbf{v})$ is a concave function. Then, (1) the set $\mathcal{Y}$ can be partitioned into $(2I_k - 1)$ subintervals, and g_c and $g_{k'}$ are piecewise-linear (affine) functions that are affine on each*

subinterval; and (2) for every $y \in \mathcal{Y}$, there exists a probability vector $\mathbf{p}(y; \mathbf{v}) \in \Delta^{I_k-1} := \{\mathbf{p} \in \mathbb{R}_+^{I_k} | \mathbf{1}^\top \mathbf{p} = 1\}$ such that at most two elements in $\mathbf{p}(y; \mathbf{v})$ are strictly positive, and that the maximized value, $m(y; \mathbf{v})$, satisfies

$$m(y; \mathbf{v}) = u(g_c(y; \mathbf{v})) + \beta \mathbf{p}(y; \mathbf{v})^\top \mathbf{v}.$$

In the following proof, instead of characterizing the policy function for a given y , I derive a condition under which the optimal saving equals a given level in the whole interval $\mathcal{K}$. This idea is derived from that of the endogenous grid method (EGM) in [Carroll \(2006\)](#), but there is a key difference: EGM only considers the values of k' on a finite, exogenous grid, whereas I consider all values in $\mathcal{K}$. This treatment is possible because of the piecewise linearity of the function $\hat{V}$, as I describe below, and it results in a global characterization of the policy function.

Proof.

[Part 1] Suppose first that $\hat{V}(k; \mathbf{v})$ is strictly increasing in k so that $(v_{i+1} - v_i)/(k_{i+1} - k_i)$ is strictly positive.

What are the levels of current cash-on-hand such that the planner finds it optimal to choose $k' \in (k_i, k_{i+1})$ for some $i = 1, \dots, I_k - 1$? Because k' is neither at a kink nor at a bound, and because $\hat{V}$ is concave, the first-order condition is necessary and sufficient:

$$u'(c) = \lambda_i := \beta \frac{v_{i+1} - v_i}{k_{i+1} - k_i} > 0.$$

The rightmost expression is strictly positive, because v_i is strictly increasing in i . Hence, the optimal consumption is given by $c = c_i^* := (u')^{-1}(\lambda_i)$. Therefore, if the current cash-on-hand y is in the open interval,

$$\mathcal{I}_i := (c_i^* + k_i, c_i^* + k_{i+1}),$$

the planner's optimal consumption and saving choices are given by $(c, k') = (c_i^*, y - c_i^*)$. In sum, the policy function satisfies

$$g_c(y; \mathbf{v}) = c_i^*, g_{k'}(y; \mathbf{v}) = y - c_i^*$$

for all $y \in \mathcal{I}_i$ and for $i = 1, \dots, I_k - 1$.

When is it optimal to choose a grid point, $k' = k_i$? First consider the boundary, $k' = k_1$ and $k' = k_{I_k}$. For $k' = k_1$ to be optimal, the first order condition requires

$$u'(c) \geq \lambda_1 := \beta \frac{v_2 - v_1}{k_2 - k_1}.$$

Hence, $c \leq c_1^*$. Therefore, if the current cash-on-hand, y , is in the interval

$$\mathcal{J}_1 := [k_1, c_1^* + k_1] \cap \mathcal{Y},$$

the planner's optimal choice for consumption and saving is given by $c = y - k_1$ and $k' = k_1$. For $k' = k_{I_k}$ to be optimal, the first order condition is

$$u'(c) \leq \lambda_{I_k-1} := \beta \frac{v_{I_k} - v_{I_k-1}}{k_{I_k} - k_{I_k-1}},$$

implying $c \geq c_{I_k-1}^*$. Therefore, if the current cash-on-hand is in the interval

$$\mathcal{J}_{I_k} := [c_{I_k-1}^* + k_{I_k}, \infty)$$

the planner finds it optimal to choose $c = y - k_{I_k}$ and $k' = k_{I_k}$.

For other kinks, the first-order condition requires

$$\lambda_i \leq u'(c) \leq \lambda_{i-1}$$

for setting $k' = k_i$, $i = 2, 3, \dots, I_k - 1$, to be optimal. It is when the current cash-on-hand is in the interval

$$\mathcal{J}_i := [c_{i-1}^* + k_i, c_i^* + k_i],$$

and the optimal choice of the planner satisfies $c = y - k_i$ and $k' = k_i$.

In sum, the policy function satisfies

$$g_c(y; \mathbf{v}) = y - k_i, g_{k'}(y; \mathbf{v}) = k_i$$

for all $y \in \mathcal{J}_i$ and for $i = 1, \dots, I_k$.

Note that I have partitioned $\mathcal{Y}$ into $2I_k - 1$ subintervals as $\mathcal{Y} = \mathcal{J}_1 \cup \mathcal{I}_1 \cup \mathcal{J}_2 \cup \dots \cup \mathcal{I}_{I_k-1} \cup \mathcal{J}_{I_k}$, and shown that the policy function is linear in each subinterval.

Suppose now that $\hat{V}(k; \mathbf{v})$ is not strictly increasing in k . Then, due to concavity, there exists $\bar{i} \geq 1$ such that $(v_{i+1} - v_i)/(k_{i+1} - k_i) \leq 0$ for all $i \geq \bar{i}$ and that $(v_{i+1} - v_i)/(k_{i+1} - k_i) > 0$ for all $i < \bar{i}$. I.e., $\hat{V}(k_{\bar{i}}; \mathbf{v}) \geq \hat{V}(k; \mathbf{v})$ for all $k \geq k_{\bar{i}}$. Because u is strictly increasing, it is never optimal to choose k' that is greater than $k_{\bar{i}}$. Hence, $k_{\bar{i}}$ can be treated effectively as an upper bound of the household's saving

choice. That is, the above proof goes through once one makes the following adjustment: if $\bar{i} > 1$, define $\mathcal{J}_{\bar{i}} = [c_{\bar{i}-1}^* + k_{\bar{i}}, \infty)$ and $\mathcal{I}_i = \mathcal{J}_{i+1} = \emptyset$ for all $i \geq \bar{i}$; and if $\bar{i} = 1$, $g_{k'}(y; \mathbf{v}) = k_1$ and $g_c(y; \mathbf{v}) = y - k_1$ for all y .

[Part 2] For $y \in \mathcal{J}_i$, let $\mathbf{p}(y; \mathbf{v})$ be a probability vector such that its i -th element is one and all other elements are zero. Then,

$$\hat{V}(g_{k'}(y; \mathbf{v}); \mathbf{v}) = \hat{V}(k_i; \mathbf{v}) = v_i = \mathbf{p}(y; \mathbf{v})^\top \mathbf{v}. \quad (1)$$

For $y \in \mathcal{I}_i$, let $\mathbf{p}(y; \mathbf{v})$ be a probability vector such that its i -th and $(i+1)$ -th elements are $(k_{i+1} - g_{k'}(y; \mathbf{v})) / (k_{i+1} - k_i)$ and $(g_{k'}(y; \mathbf{v}) - k_i) / (k_{i+1} - k_i)$, respectively, and other elements are zero. Then,

$$\hat{V}(g_{k'}(y; \mathbf{v}); \mathbf{v}) = \frac{k_{i+1} - g_{k'}(y; \mathbf{v})}{k_{i+1} - k_i} v_i + \frac{g_{k'}(y; \mathbf{v}) - k_i}{k_{i+1} - k_i} v_{i+1} = \mathbf{p}(y; \mathbf{v})^\top \mathbf{v}, \quad (2)$$

and Part 2 follows. ■

The policy functions for consumption and savings are depicted in Figure 1. They are piecewise linear. The consumption (saving) policy function is constant on $\mathcal{I}_i$ ($\mathcal{J}_i$) intervals but has a slope of one on $\mathcal{J}_i$ ($\mathcal{I}_i$, respectively) intervals.

A key advantage is that these policy functions are inexpensive to evaluate at any point $y \in \mathcal{Y}$, because evaluation of these functions does not require numerical interpolation, which is computationally costly. Once the interval containing y has been identified as $\mathcal{I}_i$, it immediately follows that the consumption policy function at y equals c_i^* , defined in the above proof of Proposition 1. Because the cash-on-hand is y , the saving policy function must be equal to the remaining resources, $y - c_i^*$. Suppose instead that y is found to be in $\mathcal{J}_i$. Then the saving policy function must be equal to k_i , and it follows that the consumption policy function must be equal to the remaining resources, $y - k_i$. Both the continuation value, $\hat{V}(g_{k'}(y; \mathbf{v}); \mathbf{v})$, and the probability vector, $\mathbf{p}(y; \mathbf{v})$, can be computed at any y , using equation (1) if $y \in \mathcal{J}_i$ for some i and equation (2) if $y \in \mathcal{I}_i$ for some i . The most computationally expensive part is to find, for each $y \in \mathcal{Y}_{grid}$, a subinterval that contains y .

Proposition 1 leads to the following algorithm to evaluate the policy and the value at multiple points $\mathcal{Y}_{grid} = \{y_1, y_2, \dots, y_{I_y}\}$ in $\mathcal{Y}$.

Algorithm 1 (Computation and Evaluation of the Analytical Solution) 1. *Compute endogenous grid points, i.e., the endpoints of all subintervals, and the policy function g_c and $g_{k'}$ on them.*

2. *For each $y \in \mathcal{Y}_{grid}$,*

(a) *Find a subinterval that contains y , and evaluate $g_c(y)$, $g_{k'}(y)$, $\mathbf{p}(y)$, and $\hat{V}(g_{k'}(y))$ at y .*

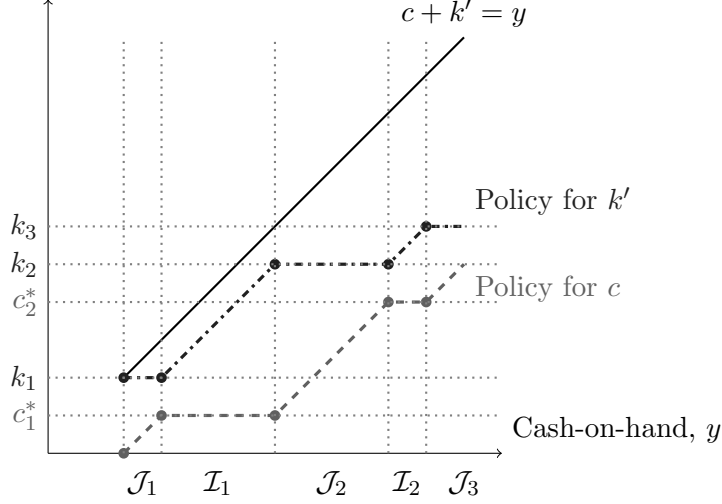


Figure 1: Policy functions for consumption and saving

(b) Compute $m(y)$ using $g_c(y)$ and $\hat{V}(g_{k'}(y))$.

To simplify the notation, I have omitted above the dependence of functions on $\mathbf{v}$. A detailed implementation of Step 1 is given in Appendix A.

Conditional on the piecewise-linear continuation value that is supplied as input, the algorithm solves the optimization problem exactly: it introduces no additional optimization or interpolation error for policy evaluation. As in the standard endogenous grid method, it is assumed that the analytical inverse of the marginal utility function is available, and obtaining $\{c_i^*\}$ — and hence the endogenous grid points — is computationally inexpensive. The algorithm does not need numerical interpolation when evaluating the policy functions and the maximized value on $\mathcal{Y}_{grid}$.

The most costly part is Step 2(a), where one needs to find, for each $y \in \mathcal{Y}_{grid}$, a subinterval that contains y . This can be done using off-the-shelf methods (e.g., MATLAB's `discretize`), but one can improve efficiency by exploiting the monotonicity of $\mathcal{Y}_{grid}$: if one has found that y_i lies between two endogenous grid points, then y_{i+1} , which is greater than y_i , cannot lie between lower endogenous grid points. Hence, one does not need to examine all subintervals when searching.

Instead of repeating Steps 2(a) and 2(b) for different y 's, one may repeat Step 2(a) for different values of y , and then obtain a vector of values, $\mathbf{m} = (m(y_1), \dots, m(y_{I_y}))^\top$, by computing

$$\mathbf{m} = \mathbf{u}(\mathbf{v}) + \beta \mathbf{P}(\mathbf{v})\mathbf{v}$$

where

$$\mathbf{u}(\mathbf{v}) = \begin{bmatrix} u(g_c(y_1; \mathbf{v})) \\ \vdots \\ u(g_c(y_{I_y}; \mathbf{v})) \end{bmatrix} \quad \text{and} \quad \mathbf{P}(\mathbf{v}) = \begin{bmatrix} \mathbf{p}(y_1; \mathbf{v})^\top \\ \mathbf{p}(y_2; \mathbf{v})^\top \\ \vdots \\ \mathbf{p}(y_{I_y}; \mathbf{v})^\top \end{bmatrix}. \quad (3)$$

The matrix $\mathbf{P}(\mathbf{v})$ is sparse, because each row has either one or two nonzero elements. There is no need for additional computation to obtain the nonzero elements of $\mathbf{P}(\mathbf{v})$: they are computed anyway to calculate $\hat{V}(g_k(\cdot))$. For the fitted value function iteration algorithm in the next section, repeating Steps 2(a) and 2(b) for different y 's is more efficient: forming a large matrix $\mathbf{P}(\mathbf{v})$ is costly and one needs to compute it every time the policy function is updated. This matrix-based evaluation is especially useful in the context of policy function iteration, because it allows us to compute efficiently the value of following a given policy function. I will come back to this issue in Section 4.

2.1 Stochastic model

The above theoretical result also holds in a stochastic environment, but the stacked vector notation becomes more complicated. Appendix B extends the model to incorporate shocks and describes the stacked vector notation. In what follows, I will stick to the deterministic environment, solely for expositional simplicity.

3 Fitted value function iteration

Proposition 1 alleviates two key drawbacks of the fitted value function iteration (VFI) with piecewise-linear interpolation. First, the analytical policy functions entirely eliminate the need for numerical optimization, which can be computationally inefficient due to kinks. Second, because the analytical policy functions are inexpensive to evaluate at many values of cash-on-hand, using a fine grid is not very costly. This is beneficial because, as shown in Judd and Solnick (1994), piecewise-linear interpolation requires a much larger grid size than Schumaker's (1983) shape-preserving interpolation to achieve the same level of accuracy. In addition, the analytical policy functions eliminate two sources of numerical error. First, because the continuation value is concave and piecewise linear in every VFI step, the optimization step is free from the optimization error, conditional on the continuation value function. Second, because the analytical policy functions are piecewise linear, their evaluation is free from policy interpolation error.

3.1 Deterministic Ramsey model

Consider a standard, deterministic neoclassical growth model in which the planner's recursive problem is given as follows. For all $k \in \mathcal{K} := [k_{min}, k_{max}]$,

$$v(k) = \max_{c, k'} \{u(c) + \beta v(k')\}$$

subject to the resource constraint,

$$c + k' \leq F(k),$$

and the constraint on the next-period capital, $k' \in \mathcal{K}$.

The preference discount factor, β , is positive and strictly less than one. The function F is assumed to be a strictly concave function satisfying $F(0) = 0$, $F'(k) > 0$, $\lim_{k \downarrow 0} F'(k) = \infty$, and $\lim_{k \uparrow \infty} F'(k) = 1 - \delta$. Later in the numerical exercise, I use a Cobb-Douglas production function, $F(k) = Ak^\alpha + (1 - \delta)k$, with $0 < \alpha < 1$ and $0 < \delta < 1$.

I now make some assumptions so that the constraint set is non-empty for all $k \in \mathcal{K}$. First, I assume that the lower bound for capital is strictly positive, $k_{min} > 0$. Second, the lower bound is not too large in that $F(k_{min}) - k_{min} > 0$. These assumptions guarantee that, for any $k \in \mathcal{K}$, a policy that sets $c = F(k) - k_{min}$ and $k' = k_{min}$ is feasible.

The true value function, v , is then bounded below by a concave function. To construct a lower bound for v , consider a suboptimal policy that always saves k_{min} . Define the function $v_{LB}(k)$ as the value from following this policy, i.e.,

$$v_{LB}(k) = u(F(k) - k_{min}) + \frac{\beta}{1 - \beta} u(F(k_{min}) - k_{min}).$$

Then $v(k) \geq v_{LB}(k) \geq v_{LB}(k_{min}) > -\infty$ for all k , and v_{LB} is concave. If setting $c = F(k) - k$ is feasible for all $k \in \mathcal{K}$, then one can instead use $v_{LB}(k) = u(F(k) - k)/(1 - \beta)$. In the numerical exercises, I use this v_{LB} as the initial condition for the VFI and PFI algorithms.

The true value function is bounded above by a constant. Define $\bar{v} \in \mathbb{R}$ by

$$\bar{v} = \frac{u(F(k_{max}) - k_{min})}{1 - \beta} < \infty.$$

Then, because the resource constraint requires $c \leq F(k) - k' \leq F(k_{max}) - k_{min}$ for any k , we have $v(k) \leq \bar{v}$ for every $k \in \mathcal{K}$.

Let B be the space of bounded, continuous functions on $\mathcal{K}$, and endow it with the sup norm, $\|\cdot\|$. Let $\mathbb{T} : B \rightarrow B$ be a Bellman operator such that, for any function $w \in B$ and for any $k \in \mathcal{K}$,

$$\mathbb{T}w(k) = \max_{c, k'} u(c) + \beta w(k')$$

subject to $c + k' \leq F(k)$, $k' \in \mathcal{K}$. Then $\mathbb{T}$ is a contraction mapping on $(B, \|\cdot\|)$, and thus has a unique fixed point in B , and it coincides with the value function, v . Moreover, by setting $v_0 := v_{LB}$ and recursively defining $v_{n+1} = \mathbb{T}v_n$, the sequence $\{v_n\}$ monotonically converges to v from below. It is straightforward to show that $\mathbb{T}$ is monotonicity- and concavity-preserving, and that v is monotone and concave.

3.2 Illustration of the VFI algorithm

The true Bellman operator cannot be exactly implemented on a computer because it requires solving the maximization problem for a continuum of values for k . Fitted value iteration methods solve the maximization problem only on a finite grid and either approximate or interpolate the resulting values over the whole domain to use it in the next optimization step.

Note that, once the cash-on-hand y is defined as $y = F(k)$, the structure of the problem is identical to that in Section 2 if the interpolant is a piecewise-linear, concave function. This observation leads to the following VFI algorithm:

Algorithm 2 (Fitted value iteration algorithm) 1. *Initialization:*

- (a) Fix the value tolerance level, $tol_v > 0$, and the capital grid, $\mathcal{K}_{grid} = \{k_1, k_2, \dots, k_{I_k}\}$ with $k_1 = k_{min}$ and $k_{I_k} = k_{max}$.
- (b) Pre-compute $\mathcal{Y}_{grid} = \{y_1, y_2, \dots, y_{I_k}\}$, with $y_i = F(k_i)$ for $i = 1, \dots, I_k$.
- (c) Set an initial value vector, $\mathbf{v}_0$, using a concave value function from a feasible policy, e.g., $v_{0,i} = v_{LB}(k_i)$ for $i = 1, \dots, I_k$.

2. *VFI: for $n = 0, \dots$, taking $\mathbf{v}_n$ as given,*

- (a) Compute the policy function on $\mathcal{Y}_{grid}$, by solving the $\mathcal{P}(y; \mathbf{v}_n)$ -problem using Algorithm 1 for all $y \in \mathcal{Y}_{grid}$.
- (b) Update the value function as $v_{n+1,i} = u(g_c(y_i; \mathbf{v}_n)) + \beta \mathbf{p}(y_i; \mathbf{v}_n)^\top \mathbf{v}_n$ for all $i = 1, \dots, I_k$.
- (c) Stop if $\|\mathbf{v}_{n+1} - \mathbf{v}_n\| < tol_v$. Go back to Step 2(a) otherwise.

Note that Step 2(b) in the above VFI algorithm is equivalent to

$$v_{n+1,i} = m(F(k_i); \mathbf{v}_n) = \max_{c,k'} u(c) + \beta \hat{V}(k'; \mathbf{v}_n),$$

where the rightmost maximization is subject to $c+k' \leq F(k_i)$ and $k' \in \mathcal{K}$, provided that $\hat{V}(k'; \mathbf{v}_n)$ is a concave function. The numerical Bellman operator $\mathbb{T}_a$ is defined by $\mathbb{T}_a \mathbf{v} := (m(F(k_1); \mathbf{v}), \dots, m(F(k_{I_k}); \mathbf{v}))^\top$, and the VFI algorithm iterates on $\mathbf{v}_{n+1} = \mathbb{T}_a \mathbf{v}_n$. It is straightforward to prove that $\mathbb{T}_a$ is a β -contraction mapping, and therefore that the iteration converges to a unique fixed point, $\mathbf{v}_*$. The mapping is concavity-preserving in the sense that $\hat{V}(k; \mathbf{v}_n)$ is concave for each n , provided that it is for $n = 0$. The algorithm produces a monotone sequence, i.e., $\mathbf{v}_{n+1} \geq \mathbf{v}_n$ for all n , given that the initial value $\mathbf{v}_0$ is taken from the value of a feasible policy. Hence, in the limit, the value vector converges to $\mathbf{v}_*$ from below.³

Effectively the above algorithm updates the value vector as

$$\mathbf{v}_{n+1} = \mathbf{u}(\mathbf{v}_n) + \beta \mathbf{P}(\mathbf{v}_n) \mathbf{v}_n,$$

where $\mathbf{u}(\mathbf{v}_n)$ and $\mathbf{P}(\mathbf{v}_n)$ are defined in equation (3). The matrix $\mathbf{P}(\mathbf{v}_n)$ is now a square matrix and can be interpreted as a probability transition matrix over $\mathcal{K}_{grid}$: all elements are nonnegative and each row sums up to one. Its sparsity is exploited in the policy function iteration, which I turn to in the next section.

Now let us evaluate the algorithm's performance in terms of computation time. As a baseline, I use the following parameterization: $\beta = 1/1.05$, $\alpha = 0.3$, $\delta = 0.05$, $A = (1/\beta - 1 + \delta)/\alpha$, and $u(c) = -c^{-1}$, i.e., $u(c) = c^{1-\sigma}/(1-\sigma)$ with $\sigma = 2$. The technology parameter A is chosen so that the steady-state capital is unity. I consider a sufficiently wide range for capital and set $k_{min} = 0.001$ and $k_{max} = 2$. Below I report the results for equally-spaced grids with 1,000 points and 10,000 points. I set the value-function convergence tolerance to 10^{-6} .

The code is written in MATLAB 2024b and is run on a MacBook Air with an Apple M3 chip and 24 GB of memory. For each grid size, I calculate the average convergence time from repeating the computation 1,000 times.

When 10,000 grid points are used, it takes on average about 0.04 seconds until convergence. In each computation, value iteration requires 73 iterations, implying that each step takes about 0.55 milliseconds. When 1,000 grid points are used instead, the average computation time is about 5.8 milliseconds. The number of value iteration steps is 72, implying that each step takes 0.08 milliseconds.

³Related convergence results for fitted value iteration with piecewise-linear or nonexpansive approximation can be found in Judd and Solnick (1994) and Stachurski (2008). Note that the numerical fixed point, $\hat{V}(\cdot; \mathbf{v}_*)$, is in general different from the true value function, v . Error bounds established in these papers are applicable under their respective assumptions.

Table 1: Computation time: VFI

	$I_k = 10,000$	$I_k = 1,000$
Total time (seconds)	3.945×10^{-2}	5.790×10^{-3}
Number of steps	73	72
Time per iteration step (seconds)	5.405×10^{-4}	8.041×10^{-5}

As discussed in the introduction, [Barillas and Fernandez-Villaverde \(2007\)](#) also combine the standard EGM with VFI to improve efficiency. In each optimization step, they apply the standard EGM using an estimate of the continuation value derivative on a finite grid, and obtain optimal consumption and saving choices as well as the maximized value on an endogenous grid. These values are then interpolated (and extrapolated if necessary) to evaluate policy and value functions on an exogenous grid. Hence, conditional on the value function estimate, their method solves the optimization problem only approximately and interpolation may introduce policy-evaluation error. In contrast, the analytical policy function in the present paper solves the optimization problem exactly at any value of cash-on-hand, conditional on a concave piecewise-linear value function.

4 Policy function iteration

Further acceleration is possible through policy function iteration (PFI) algorithms.⁴ One advantage of using piecewise-linear interpolation is that standard PFI reduces exactly to solving a sparse linear system of equations, and modified PFI reduces to repeated sparse matrix multiplications. [Rendahl \(2022\)](#) demonstrates that, once the policy function for saving is approximated by a probability transition matrix over a discretized state space, its sparsity can be effectively exploited for PFI to accelerate convergence. For the approximated policy function, PFI amounts to solving a sparse linear system of equations. He uses [Young’s \(2010\)](#) “lottery” or “histogram” method to obtain such approximation for a continuous policy function.⁵

The present paper complements [Rendahl \(2022\)](#) in four respects. First, it provides an analytical policy function for a piecewise-linear value function, substantially accelerating the optimization step, on top of the speed gain that can be achieved by his matrix-based policy function iteration algorithm. Second, in the present paper’s method, the probability transition matrix in step n equals the $\mathbf{P}(\mathbf{v}_n)$ matrix, which can be formed using the interpolation weights that are computed anyway for VFI, whereas in general [Young’s](#)

⁴The concept of policy function iteration originates from [Bellman’s \(1957\)](#) approximation in policy space and [Howard’s \(1960\)](#) improvement algorithm.

⁵[Young \(2010\)](#) originally uses it to compute a wealth distribution in a heterogeneous-agent model. Hence, [Rendahl \(2022\)](#) shows that the same matrix can serve for two purposes in heterogeneous-agent models.

(2010) method requires an additional computational step that approximates a continuous policy function with a transition matrix. Third, because the proposed method uses piecewise-linear value functions, it is guaranteed that there is no approximation error in the continuation value evaluation from using a probability transition matrix over a finite grid, instead of a continuous policy function, in the policy evaluation steps.⁶ This is advantageous because, while such approximation is possible for general policy functions, error bounds for approximation are not established. Fourth, the present paper uses concavification of the value function to achieve stability, because PFI in general does not preserve concavity of the value function.

Let $\mathbf{u}_n = \mathbf{u}(\mathbf{v}_n)$ and $\mathbf{P}_n = \mathbf{P}(\mathbf{v}_n)$. Recall that $\mathbf{P}_n$ is a sparse probability matrix because each row of $\mathbf{P}_n$ has either one or two nonzero elements.

Define an operator $\mathbb{T}_{p,n}$ by

$$\mathbb{T}_{p,n}\mathbf{w} = \mathbf{u}_n + \beta\mathbf{P}_n\mathbf{w}.$$

Standard PFI computes, at each step n , the value of permanently following the policy function computed in step n . The value on the asset grid, $\mathbf{w}_n$, is then a fixed point of $\mathbb{T}_{p,n}$, i.e., $\mathbf{w}_n = \mathbb{T}_{p,n}\mathbf{w}_n$. The solution to this equation is obtained by solving a sparse linear system of equations $(\mathbf{I} - \beta\mathbf{P}_n)\mathbf{w}_n = \mathbf{u}_n$ for $\mathbf{w}_n$. The coefficient matrix $\mathbf{I} - \beta\mathbf{P}_n$ is nonsingular, because $\mathbf{P}_n$ is a probability transition matrix and $\beta < 1$, and thus the fixed point is unique.

Modified PFI (MPFI) is indexed by a pre-specified integer $J \geq 1$ and instead iterates $\mathbf{w}_n^{(j+1)} = \mathbb{T}_{p,n}\mathbf{w}_n^{(j)}$, starting from $j = 0$ and $\mathbf{w}_n^{(0)} = \mathbf{v}_n$, until j reaches $J - 1$ to obtain $\mathbf{w}_n^{(J)}$. The idea is that $\mathbf{w}_n^{(J)}$ is a good approximation of $\mathbf{w}_n$, because $\mathbf{w}_n^{(J)} \rightarrow \mathbf{w}_n$ as $J \rightarrow \infty$. Hence, modified PFI requires repeated sparse matrix multiplications.

Note, however, that neither $\mathbf{w}_n$ nor $\mathbf{w}_n^{(J)}$ for $\mathbf{v}_{n+1}$ cannot be directly used as $\mathbf{v}_{n+1}$ in the next policy-improvement (or optimization) step, because neither of their piecewise-linear interpolants is guaranteed to be concave. If $\hat{V}(\cdot; \mathbf{v}_{n+1})$ fails to be concave, then Proposition 1 is not applicable. This problem can be circumvented by choosing $\mathbf{v}_{n+1}$ so that it concavifies these interpolants. For concavification, let us define a concavification operator, $\mathbb{C}$, such that $\mathbb{C}\mathbf{w}$ is the vector of nodal values of the least concave majorant of $\hat{V}(\cdot; \mathbf{w})$, i.e., $\hat{V}(\cdot; \mathbb{C}\mathbf{w})$ is the smallest concave function with $\hat{V}(k; \mathbb{C}\mathbf{w}) \geq \hat{V}(k; \mathbf{w})$ for all $k \in \mathcal{K}$. Figure 2 illustrates the concavification operator.⁷

The PFI algorithm replaces Step 2(b) in the VFI algorithm with

(b') Compute $\mathbf{w}_n$, and update the value vector so that $\mathbf{v}_{n+1} = \mathbb{C}\mathbf{w}_n$.

⁶The absence of approximation error under piecewise-linear value functions is noted in Rendahl (2022).

⁷There are numerical algorithms for constructing $\mathbb{C}\mathbf{w}$ from $\mathbf{w}$. I used the monotone chain algorithm by Andrew (1979). Although the algorithm is originally designed to compute a convex hull of a set of points in $\mathbb{R}^2$, it can be easily modified to compute only the upper concave envelope.

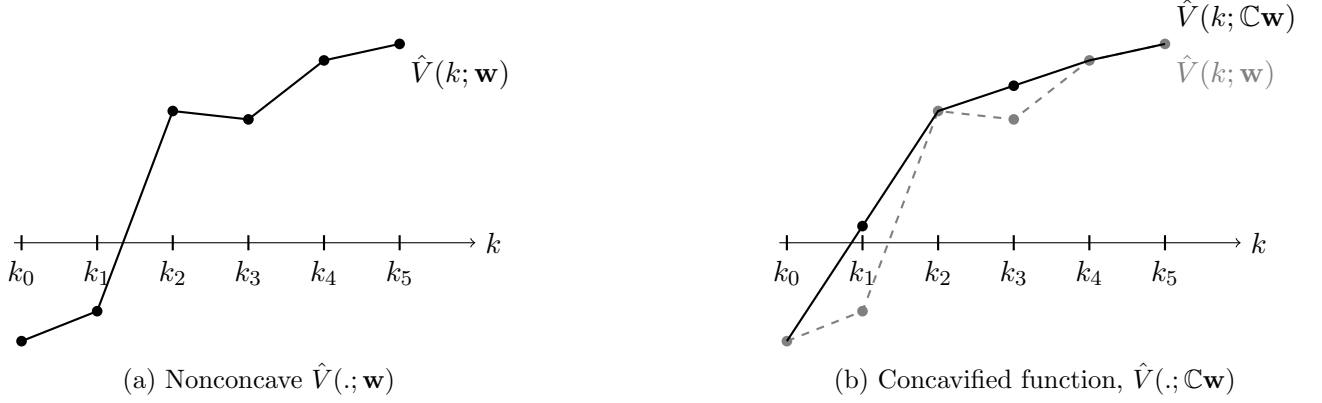


Figure 2: Illustration of concavification

Table 2: Computation time comparison: VFI vs. PFI

	$I_k = 10,000$			$I_k = 1,000$		
	VFI	PFI	MPFI ($J = 20$)	VFI	PFI	MPFI ($J = 20$)
Total time (sec.)	3.945×10^{-2}	1.379×10^{-2}	9.343×10^{-3}	5.790×10^{-3}	1.476×10^{-3}	1.204×10^{-3}
Number of steps	73	8	8	72	7	7
Time per step (sec.)	5.405×10^{-4}	1.724×10^{-3}	1.168×10^{-3}	8.041×10^{-5}	2.109×10^{-4}	1.720×10^{-4}

The modified PFI algorithm is identical to the PFI algorithm except that it computes $\mathbf{w}_n^{(J)}$ and sets $\mathbf{v}_{n+1} = \mathbb{C}\mathbf{w}_n^{(J)}$.

Both PFI and MPFI converge to the same limit as VFI, $\mathbf{v}_*$, due to their monotonicity. The key is to use the value of a feasible policy as $\mathbf{v}_0$. Let $\{\mathbf{v}_n^{VFI}\}$ and $\{\mathbf{v}_n^{PFI}\}$ be sequences of value vectors under VFI and PFI, respectively, with $\mathbf{v}_0^{VFI} = \mathbf{v}_0^{PFI} = \mathbf{v}_0 \leq \mathbf{v}_*$. Then for each $n \geq 1$, we have $\mathbf{v}_* \geq \mathbf{v}_n^{PFI} \geq \mathbf{v}_n^{VFI}$.⁸ Because the rightmost term converges to $\mathbf{v}_*$ as $n \rightarrow \infty$, so does $\mathbf{v}_n^{PFI}$. The convergence of the value vector sequence under MPFI can be shown in the same manner.

When 10,000 grid points are used, PFI takes on average 0.014 seconds until convergence, which is about one-third of computation time of VFI. In each computation, PFI requires 8 iterations until convergence, implying that each iteration takes about 1.7 milliseconds. When 1,000 grid points are used instead, the average computation time is 1.5 milliseconds. The number of iteration steps is 7, implying that each step takes 0.21 milliseconds.

PFI achieves a speed gain relative to VFI by substantially reducing the number of iteration steps. For 10,000 grid points, the number of iterations decreases from 73 to 8. For 1,000 grid points, it decreases from 72 to 7. The computation time per iteration step is longer for PFI than for VFI, because PFI computes the

⁸Proof is by induction. It is clear that these inequalities hold for $n = 1$. Suppose that they hold for n . Then, $\mathbf{v}_* \geq \mathbf{v}_{n+1}^{PFI} \geq \mathbb{T}_{p,n}\mathbf{v}_n^{PFI} = \mathbb{T}_a\mathbf{v}_n^{PFI} \geq \mathbb{T}_a\mathbf{v}_n^{VFI} = \mathbf{v}_{n+1}^{VFI}$. Hence, these inequalities also hold for $n + 1$.

value of following the analytical policy function and concavifies the resulting value function. Yet, the speed gain from a reduction in the number of iteration steps dominates, and the total computation time is cut by 50-70%. Note that the PFI results in Table 2 use a MEX function to speed up the concavification step, but PFI remains faster than VFI, even without a compiled implementation.⁹

The modified PFI with $J = 20$ slightly outperforms the standard PFI for the model considered here. Compared to PFI, MPFI is about 30% faster for 10,000 grid points and about 20% for 1,000 grid points.¹⁰

Overall, PFI and MPFI algorithms can achieve substantial acceleration. However, piecewise-linear interpolation may suffer from approximation error when the curvature of the value function is large. Section 6 attempts to improve accuracy.

5 Speed comparison with other methods

In this section I compare the above method with two widely-used methods that are known to be fast: the standard endogenous grid method (EGM) and the continuous-time method.

5.1 Comparison with the standard EGM

The standard EGM has been widely used to solve discrete-time income-fluctuation problems. The main reason is its speed.

In terms of speed, the standard EGM has an advantage over the VFI algorithm in this paper. The standard EGM is a fixed point iteration, and during iteration it does not compute the value function, which converges only slowly. Instead, it uses the envelope theorem to update an estimate of the derivative of the value function. Therefore, it is expected that, when the EGM converges, it converges faster than the VFI. However, it is not obvious how fast it is relative to the VFI. It is also not clear whether the standard EGM is faster than the PFI algorithm in this paper.

Comparing their speeds is not straightforward. Because the standard EGM does not compute the value function, one cannot use the same stopping criterion as the VFI algorithm. To make a fair comparison, I compute in the VFI algorithm how much the consumption policy function is updated on the exogenous grid in the final iteration step, $\max_{k \in \mathcal{K}_{grid}} |g_{c,n}(k) - g_{c,n-1}(k)|$, and use it as the stopping criterion for the standard EGM. The same initial condition is used for the standard EGM as for the VFI, implying that the

⁹Without a MEX function, computation time increases to 2.841×10^{-2} for 10,000 grid points and 2.634×10^{-3} for 1,000 grid points.

¹⁰MPFI also use a MEX function to accelerate the concavification step. Without a MEX function, computation time increases to 2.481×10^{-2} for 10,000 grid points and 2.433×10^{-3} for 1,000 grid points.

Table 3: Computation time comparison: VFI, PFI vs. EGM

	EGM	VFI	PFI	MPFI($J = 20$)
Total time (sec.)	3.001×10^{-2}	3.945×10^{-2}	1.379×10^{-2}	9.343×10^{-3}
Number of steps	72	73	8	8

initial estimate of the value function derivative is given by

$$Dv_{0,i} = \frac{u'(F(k_i) - k_i)(F'(k_i) - 1)}{1 - \beta}$$

for all $i = 1, 2, \dots, I_k$.¹¹ I set the grid size $I_k = 10,000$. It turns out that the standard EGM takes 72 steps — one step less than VFI — until the consumption policy function satisfies the aforementioned stopping criterion. Appendix C describes the standard EGM algorithm in detail.

Table 3 compares the computation time for the standard EGM, along with that for VFI and PFI. The standard EGM converges about 30% faster than the VFI method proposed here. This property reflects the fact that the number of endogenous grid points for the standard EGM equals I_k whereas that for the VFI method equals $2I_k$, making it more costly to evaluate the interpolant on the exogenous grid. Note that, if one needs to compute the value function (e.g., for welfare calculation) too, then the above speed comparison may not be fair: the VFI figure includes the computation time for the value function but the EGM figure does not. The overall computation time difference is expected to be much smaller when the value calculation is necessary. Moreover, with a smaller grid size of $I_k = 1,000$, VFI converges as fast as EGM: VFI takes 72 steps and 5.790×10^{-3} seconds to converge, whereas EGM takes 73 steps and 5.868×10^{-3} seconds.

Because both PFI and MPFI algorithms significantly accelerate convergence, they are much faster than the standard EGM, even though these algorithms compute the value function, which EGM does not. They perform equally well with a smaller grid size of $I_k = 1,000$, with PFI taking 1.476×10^{-3} seconds and MPFI taking 1.204×10^{-3} seconds to converge. Therefore, the proposed method in the present paper is particularly effective for infinite-horizon, stationary problems, for which PFI and MPFI are applicable, and useful for applications that require welfare calculation.

Another point made in Rendahl (2022) is that the sparse probability transition matrix used for PFI can be used to compute a stationary distribution of assets in a heterogeneous-agents model, as Achdou et al. (2021) show for the continuous-time model. Given that the proposed method improves the speed in the optimization step, it is worthwhile to examine how fast a stationary equilibrium in a discrete-time

¹¹This is the derivative of the initial value function, $u(F(k) - k)/(1 - \beta)$, in the VFI algorithm.

Table 4: Computation time for the deterministic growth model: discrete-time PFI vs. continuous-time implicit method

	$I_k = 10,000$			$I_k = 1,000$		
	PFI	MPFI	Cont.-time	PFI	MPFI	Cont.-time
Total time (sec.)	1.379×10^{-2}	9.343×10^{-3}	1.056×10^{-2}	1.476×10^{-3}	1.204×10^{-3}	2.013×10^{-3}
Number of steps	8	8	6	7	7	7
Time per step (sec.)	1.724×10^{-3}	1.168×10^{-3}	1.751×10^{-3}	2.109×10^{-4}	1.720×10^{-4}	2.875×10^{-4}

heterogeneous-agents model can be computed using the proposed algorithms, relative to its continuous-time counterpart. Appendix D examines this issue.

In Section 6, I compare the Euler equation error generated by the method proposed in the present paper and the standard EGM. Although the standard EGM achieves lower Euler equation error, there is a simple modification for the proposed method that makes the Euler equation error comparable to that of the standard EGM.

5.2 Comparison with the continuous-time method

Note that the method presented here resembles that of a continuous-time heterogeneous-agents model in Achdou et al. (2021): (1) the policy function is computed using finite differences of the value function, and (2) the PFI is implemented using sparse linear algebra. Because the continuous-time method is often viewed as a fast method, it makes sense to compare the present paper’s method with it.

In this section, I compare two methods using the deterministic growth model. The continuous-time benchmark is the implicit method, and I use Ben Moll’s code written in MATLAB.¹²¹³ Table 4 reports the results, both for the grid size of 10,000 and of 1,000, along with the PFI results in the discrete-time model. When running these files, I disable all commands that display variables and that create figures.

When the grid size is 1,000, both PFI and MPFI algorithms are slightly faster than the continuous-time counterpart. For the continuous-time method, the number of steps is 7 and the average computation time is 2 milliseconds, implying that each step takes 0.29 milliseconds. Although the number of iterations is the same as PFI and MPFI, PFI is about 25% faster and MPFI is about 40% faster than the continuous-time implicit method. These numbers suggest that both PFI and MPFI methods in the present paper performs well for a grid size that is not too large.

With a larger grid size, however, the continuous-time method is faster than PFI. When the grid size is as

¹²http://benjaminmoll.com/wp-content/uploads/2020/06/HJB_NGM_implicit.m, last access: September 22, 2026.

¹³His parameterization is identical to mine, except that the technology A is not normalized so that the steady state capital is not equal to one. The lower and upper bounds for capital are set to $0.001k_{ss}$ and $2k_{ss}$, respectively.

large as 10,000, the total computation time for the continuous-time method is about 25% less than that for the discrete-time PFI method. However, this difference arises entirely from the difference in the number of iterations until convergence: the number of iteration steps is smaller by about 25% for the continuous-time method than for PFI, whereas the time per iteration step is almost identical across the two methods. It is noteworthy that MPFI slightly outperforms the continuous-time method even though the former requires more iterations until convergence than the latter. The computation time per iteration step is about 33% smaller for MPFI than that for the continuous-time method. Hence, the continuous-time implicit method may benefit from applying the idea of MPFI, which is to repeat sparse matrix multiplications for finite times instead of solving a sparse system of linear equations.

Apart from the computational cost of concavification, there is a reason why the method proposed in this paper is slightly slower than the continuous-time counterpart. It is that there is an additional constraint in a continuous-time model: saving is a continuous function of time. In the continuous-time model, if the decision-maker is currently on a grid point k_i , then within a small amount of time, she either moves to a subinterval below, (k_{i-1}, k_i) , or moves to a subinterval above, (k_i, k_{i+1}) , or stays at k_i . It is simply infeasible for the decision-maker on a grid point to move to nonadjacent subintervals. In contrast, in a discrete-time model the decision-maker does not face such a constraint. Therefore, the decision-maker on each saving grid point may choose to move to nonadjacent subintervals or to another grid point. When solving the income fluctuation problem in a discrete-time model, one needs to take care of these possibilities, thereby increasing the computational costs.¹⁴

6 Improving the Euler equation error

This section compares the Euler equation errors generated by the method proposed in this paper and the standard EGM, and introduces a simple modification that substantially improves the accuracy of the proposed method.

To define the Euler equation error, let $c(k) := g_c(F(k))$ and $k'(k) := g_k(F(k))$ be the optimal current consumption and next-period capital and $c'(k) := g_c(F(k'(k)))$ be the optimal next-period consumption as functions of current capital. The Euler equation error is then defined as

$$\max_{k \in \mathcal{K}_{grid}} \left| \frac{(u')^{-1} [\beta u'(c'(k)) F'(k'(k))]}{c(k)} - 1 \right|.$$

¹⁴The initial condition assumed here, under which k is held constant, is also more advantageous to the continuous-time method. It is expected to be close to the true solution in the continuous-time model, because by assumption capital cannot change much in a short period of time. However, in the discrete-time model, the true solution may not be as close to the initial condition.

Table 5: Euler equation error

PFI	Standard EGM	PFI + 1-step EGM	PFI + 2-step EGM	PFI + 3-step EGM
6.068×10^{-3}	1.343×10^{-4}	9.652×10^{-4}	1.618×10^{-4}	1.223×10^{-4}

Columns 1 and 2 of Table 5 report the Euler equation error for the method in the present paper and the standard EGM, using the grid size $I_k = 10,000$. The proposed method yields the Euler equation error that is about 45 times larger than that of the standard EGM. This outcome is expected, given that the method in the present paper uses piecewise-linear interpolation for the value function, which is known to suffer from large error where the curvature of the value function is large. The standard EGM achieves lower error, because it uses a condition not employed in the method in the present paper: the envelope condition. The envelope condition provides an accurate estimate for the derivative of the value function, whereas the method in the present paper effectively uses a finite-difference approximation, which is much less accurate when the curvature is large.

However, the difference between the consumption policy functions obtained by the two methods is actually small: the maximal difference on the grid is 5.40×10^{-5} in absolute value, and the percentage difference is 0.16%. This suggests that the difference does not matter quantitatively for computing, for example, consumption dynamics and that a small modification of the policy function obtained by the method in the present paper can significantly reduce the Euler equation error.¹⁵

A simple modification yields substantial improvement in accuracy: after obtaining the policy function using the PFI method, apply two steps of the standard EGM (or three steps if necessary). This *hybrid* approach is computationally inexpensive yet retains benefits of both methods: (1) it is as fast as the proposed method because PFI accelerates convergence, (2) its convergence is guaranteed; and (3) it utilizes the envelope condition to reduce the Euler equation error. The fourth and fifth column in Table 5 reports the Euler equation error for this approach. As a result of this modification, the Euler equation error becomes comparable to that of the standard EGM. It appears that applying the EGM step just once is not enough, as shown in the third column in Table 5.

These findings suggest that the present paper’s method can be easily modified so that the Euler equation error is comparable to the standard EGM while maintaining its advantages. There are other well-known remedies that can be applied here, on top of the above modification. The error can be made smaller by using a log-spaced grid, which puts more points near the lower end of the interval $\mathcal{K}$. The error can also be

¹⁵Consumption policy functions obtained by VFI, PFI, and MPFI are very close to each other. Their maximal differences on the grid are about 3×10^{-7} .

reduced by raising the lower bound, k_{min} .

7 Conclusion

In this paper, I show that an analytical solution exists for income fluctuation problems with one asset if the value of saving is a concave piecewise-linear function. The analytical solution is useful in various ways. First, because it is computationally inexpensive to obtain and to evaluate, it accelerates the value function iteration algorithm based on piecewise-linear interpolation, by removing its biggest bottleneck — kinks in the objective function prevent us from using efficient numerical optimization algorithms that require a smooth objective function. Second, the analytical policy function can further accelerate convergence using the policy function iteration algorithm proposed by [Rendahl \(2022\)](#). As a result of these acceleration schemes, both VFI and PFI algorithms are comparable in terms of speed to existing methods that are known to be fast: the standard endogenous grid method and the continuous-time implicit method. Accuracy as measured by the Euler equation error can be improved by a simple modification.

Given the increasing relevance of income fluctuation problems with multiple assets (e.g. liquid and illiquid assets, short-term and long-term assets, etc.), it is of interest to generalize the result in the present paper to a multidimensional setting. This extension is left for future research.

References

- ACHDOU, YVES, JIEQUN HAN, JEAN-MICHEL LASRY, PIERRE-LOUIS LIONS, AND BENJAMIN MOLL (2021): “Income and Wealth Distribution in Macroeconomics: A Continuous-Time Approach,” *The Review of Economic Studies*, 89, 45–86.
- ANDREW, A.M. (1979): “Another efficient algorithm for convex hulls in two dimensions,” *Information Processing Letters*, 9, 216–219.
- AUCLERT, ADRIEN, BENCE BARDÓCZY, MATTHEW ROGNLIE, AND LUDWIG STRAUB (2021): “Using the Sequence-Space Jacobian to Solve and Estimate Heterogeneous-Agent Models,” *Econometrica*, 89, 2375–2408.
- BARILLAS, FRANCISCO AND JESUS FERNANDEZ-VILLAYERDE (2007): “A generalization of the endogenous grid method,” *Journal of Economic Dynamics and Control*, 31, 2698–2712.

- BELLMAN, RICHARD (1957): *Dynamic Programming*, Princeton, NJ: Princeton University Press, dover (2003) paperback edition.
- CAI, YONGYANG AND KENNETH L. JUDD (2012): “Shape-preserving dynamic programming,” *Mathematical Methods of Operations Research*, 77, 407–421.
- (2014): “Chapter 8 - Advances in Numerical Dynamic Programming and New Applications,” in *Handbook of Computational Economics Vol. 3*, ed. by K. Schmedders and K. L. Judd, Elsevier, vol. 3 of *Handbook of Computational Economics*, 479–516.
- CARROLL, CHRISTOPHER D. (2006): “The method of endogenous gridpoints for solving dynamic stochastic optimization problems,” *Economics Letters*, 91, 312–320.
- FUKUSHIMA, KENICHI AND YUICHIRO WAKI (2013): “A polyhedral approximation approach to concave numerical dynamic programming,” *Journal of Economic Dynamics and Control*, 37, 2322–2335.
- HOWARD, RONALD A. (1960): *Dynamic Programming and Markov Process*, Cambridge, MA.: The MIT Press.
- HUGGETT, MARK (1993): “The risk-free rate in heterogeneous-agent incomplete-insurance economies,” *Journal of Economic Dynamics and Control*, 17, 953–969.
- JUDD, KENNETH L. (1998): *Numerical Methods in Economics*, Cambridge, MA.: The MIT Press.
- JUDD, KENNETH L. AND ANDREW SOLNICK (1994): “Numerical Dynamic Programming with Shape-Preserving Splines,” *mimeo*.
- PUTERMAN, MARTIN L. AND MOON CHIRL SHIN (1978): “Modified Policy Iteration Algorithms for Discounted Markov Decision Problems,” *Management Science*, 24, 1127–1137.
- REND AHL, PONTUS (2022): “Continuous vs. discrete time: Some computational insights,” *Journal of Economic Dynamics and Control*, 144.
- SANTOS, MANUEL S. AND JESUS VIGO-AGUIAR (1998): “Analysis of a Numerical Dynamic Programming Algorithm Applied to Economic Models,” *Econometrica*, 66, 409–426.
- SCHUMAKER, LARRY I. (1983): “On Shape Preserving Quadratic Spline Interpolation,” *SIAM Journal on Numerical Analysis*, 20, 854–864.

STACHURSKI, JOHN (2008): “Continuous State Dynamic Programming via Nonexpansive Approximation,” *Computational Economics*, 31, 141–160.

YOUNG, ERIC R. (2010): “Solving the incomplete markets model with aggregate uncertainty using the Krusell-Smith algorithm and non-stochastic simulations,” *Journal of Economic Dynamics and Control*, 34, 36–41.

8 Statements and Declarations

This work was supported by Japan Society for the Promotion of Science (JSPS) KAKENHI Grant-in-Aid for Scientific Research (A) Grant Number 22H00058. The author has no relevant financial or non-financial interests to disclose.

Appendix

A Implementation of Step 1 in Algorithm 1

The endogenous grid points $\{y_1^{end}, \dots, y_{2I_k}^{end}\}$ are given by:

$$\begin{aligned} y_1^{end} &= k_1, \\ y_{2i}^{end} &= c_i^* + k_i, \text{ for } i = 1, \dots, I_k - 1, \\ y_{2i+1}^{end} &= c_i^* + k_{i+1}, \text{ for } i = 1, \dots, I_k - 1, \\ y_{2I_k}^{end} &= y_{max}, \end{aligned}$$

where y_{max} is a constant that is strictly greater than both $c_{I_k-1}^* + k_{I_k}$ and the largest *exogenous* grid point, y_{I_y} . In theory, y_{max} can be arbitrarily large, but in practice it is better to set it to a number that is close to $\max\{c_{I_k-1}^* + k_{I_k}, y_{I_y}\}$.

The policy function for consumption takes on the following values on the endogenous grid:

$$\begin{aligned} g_c(y_1^{end}) &= 0, \\ g_c(y_{2i}^{end}) &= g_c(y_{2i+1}^{end}) = c_i^*, \text{ for } i = 1, \dots, I_k - 1, \\ g_c(y_{2I_k}^{end}) &= y_{max} - k_{I_k}. \end{aligned}$$

Note that $g_c(y_1^{end}) = 0$ does not conflict with $c > 0$, because $y > y_1^{end} = k_{min}$ for any $y \in \mathcal{Y}$ by assumption. The policy function for saving on the endogenous grid can be obtained by calculating $g_k(y_i^{end}) = y_i^{end} - g_c(y_i^{end})$ for all i .

B Stochastic model

Imagine that a shock, z , follows a time-homogeneous, first-order Markov chain, with the set of possible values denoted by Z and the transition probability from z to z' by $\pi_{z,z'}$. Let $\mathbf{\Pi}$ be the transition matrix, with its (i, j) -element being π_{z_i, z_j} . Let N_z be the size of Z .

The grid for current cash-on-hand is shock-dependent and denoted by $\mathcal{Y}_{grid}(z) = \{y_1(z), \dots, y_{I_y}(z)\}$. For simplicity I assume that the grid size is common for all z , but it is not necessary.

Consider the following consumption-saving problem with uncertainty: for each z and each $y \in \mathcal{Y}_{grid}(z)$,

$$m(y, z; \mathbf{v}(\cdot)) := \max_{c, k'} \{u(c) + \beta \sum_{z' \in Z} \pi_{z, z'} \hat{V}(k'; \mathbf{v}(z'))\}$$

subject to the budget constraint

$$c + k' \leq y,$$

and the constraint on next-period capital, $k' \in \mathcal{K}$. Here, $\mathbf{v}(\cdot)$ is a function such that, for each z' , $\mathbf{v}(z')$ is a value vector conditional on the next-period shock being z' .

We assume that for each $z' \in Z$, $\hat{V}(k'; \mathbf{v}(z'))$ is a concave piecewise-linear function. Then, for each z , the conditional expected value, $\sum_{z' \in Z} \pi_{z, z'} \hat{V}(k'; \mathbf{v}(z'))$, in the objective function is also a concave piecewise-linear function with a value vector $\sum_{z'} \pi_{z, z'} \mathbf{v}(z')$. In other words,

$$\sum_{z'} \pi_{z, z'} \hat{V}(k'; \mathbf{v}(z')) = \hat{V}(k'; \mathbf{v}^e(z)),$$

where $\mathbf{v}^e(z) := \sum_{z'} \pi_{z, z'} \mathbf{v}(z')$.

It is therefore obvious that, for a given z , Proposition 1 can still be used to characterize the solution to the maximization problem.

Because of the presence of the transition matrix $\mathbf{\Pi}$, the stacked vector notation needs be changed as follows:

$$\mathbf{m} = \mathbf{u} + \beta \tilde{\mathbf{P}} \mathbf{v},$$

where

$$\begin{aligned} \mathbf{m} &= \begin{bmatrix} \mathbf{m}(z_1) \\ \vdots \\ \mathbf{m}(z_{N_z}) \end{bmatrix} \quad \text{with } \mathbf{m}(z) = \begin{bmatrix} m(y_1(z), z), \\ \vdots \\ m(y_{I_y}(z), z) \end{bmatrix} \\ \mathbf{v} &= \begin{bmatrix} \mathbf{v}(z_1) \\ \vdots \\ \mathbf{v}(z_{N_z}) \end{bmatrix} \quad \text{with } \mathbf{v}(z) = \begin{bmatrix} v(k_1, z), \\ \vdots \\ v(k_{I_k}, z) \end{bmatrix}, \\ \mathbf{u} &= \begin{bmatrix} \mathbf{u}(z_1) \\ \vdots \\ \mathbf{u}(z_{N_z}) \end{bmatrix} \quad \text{with } \mathbf{u}(z) = \begin{bmatrix} u(g_c(y_1(z); \mathbf{v}^e(z))) \\ \vdots \\ u(g_c(y_{I_y}(z); \mathbf{v}^e(z))) \end{bmatrix}, \end{aligned}$$

and

$$\tilde{\mathbf{P}} := \begin{bmatrix} \mathbf{\Pi}_{1,\cdot} \otimes \mathbf{P}(z_1) \\ \mathbf{\Pi}_{2,\cdot} \otimes \mathbf{P}(z_2) \\ \vdots \\ \mathbf{\Pi}_{N_z,\cdot} \otimes \mathbf{P}(z_{N_z}) \end{bmatrix} \quad \text{with } \mathbf{P}(z) = \begin{bmatrix} \mathbf{p}(y_1(z); \mathbf{v}^e(z))^\top \\ \vdots \\ \mathbf{p}(y_{I_y}(z); \mathbf{v}^e(z))^\top \end{bmatrix}$$

for all $z \in \{z_1, \dots, z_{N_z}\}$, and $\mathbf{\Pi}_{i,\cdot}$ denotes the i -th row of the matrix $\mathbf{\Pi}$ for all $i = 1, \dots, N_z$.

C Standard EGM algorithm

Algorithm 3 (Standard EGM algorithm) 1. *Initialization:*

- (a) Fix the policy tolerance level, $\text{tol}_c > 0$, and the capital grid, $\mathcal{K}_{\text{grid}} = \{k_1, k_2, \dots, k_{I_k}\}$ with $k_1 = k_{\min}$ and $k_{I_k} = k_{\max}$.
- (b) Pre-compute $\mathcal{Y}_{\text{grid}} = \{y_1, y_2, \dots, y_{I_k}\}$, with $y_i = F(k_i)$ for $i = 1, \dots, I_k$.
- (c) Set an initial condition for the value function derivative estimate, $\mathbf{Dv}_0$, so that $Dv_{0,i} = u'(F(k_i) - k_i)(F'(k_i) - 1)/(1 - \beta)$ for $i = 1, \dots, I_k$.

2. *EGM:* for $n = 0, \dots$, taking $\mathbf{Dv}_n$ as given,

- (a) Compute, for all $i = 1, \dots, I_k$, $c_i^{**} := (u')^{-1}(\beta Dv_{n,i})$ and $y_i^{\text{end}} = c_i^{**} + k_i$.
- (b) Obtain the policy functions for consumption and capital, $g_{c,n}(y_i)$ and $g_{k,n}(y_i)$, by linearly interpolating data $\{y_j^{\text{end}}, c_j^{**}\}$ and $\{y_j^{\text{end}}, k_j\}$, respectively.
- (c) Update the value function derivative estimate as $Dv_{n+1,i} = u'(g_{c,n}(y_i))F'(k_i)$ for all $i = 1, \dots, I_k$.

(d) Stop if $\max_i |g_{c,n}(y_i) - g_{c,n-1}(y_i)| < \text{tol}_c$. Go back to Step 2(a) otherwise.

D Heterogeneous-agent modelling

Both VFI and PFI algorithms in a stochastic model are straightforward, given stacked vector notation in Appendix B.

D.1 Incomplete-market, heterogeneous-agents model

Consider the following household's problem in the [Huggett \(1993\)](#) model in a stationary equilibrium:

$$v(a, z) = \max_{c, a' \leq a_{max}} u(c) + \beta \sum_{z' \in Z} \pi_{z, z'} v(a', z')$$

subject to the budget constraint,

$$c + a' \leq (1 + r)a + z,$$

and the borrowing constraint,

$$a' \geq a_{min}.$$

The household faces a constant real interest rate, r , and, in each period, chooses its consumption, c , and the next period asset holding, a' , after receiving an exogenous endowment, z . The endowment z evolves according to a finite-state Markov chain with the state space Z . Let z_{min} denote the smallest element of Z , and I assume that $z_{min} > 0$. The household's preference discount factor β satisfies $0 \leq \beta < 1$. The real interest rate, r , is assumed to be strictly positive, $r > 0$, and to satisfy $\beta(1 + r) < 1$. I assume that borrowing is allowed, $a_{min} < 0$, but that the borrowing limit is tight in that $a_{min} > -z_{min}/r$. It is also assumed that $a_{max} > 0$.

Because the borrowing limit is tight and $r > 0$, it is feasible to set $a' = a$ at any current asset level $a \geq a_{min}$: with $a' = a$, the maximal possible consumption level is $(1 + r)a - a + z = ra + z \geq ra_{min} + z = r(a_{min} + z_{min}/r) > 0$. The value of following such a policy can be used as the initial condition for VFI and PFI.

For a given real interest rate, this income fluctuation problem fits the previous stochastic growth model framework by defining $a = k$ and $F(k, z) = (1 + r)k + z$. Hence, the VFI and the PFI algorithms approximately obtain $\mathbf{v}(r) = \mathbf{u}(\mathbf{v}(r)) + \beta \tilde{\mathbf{P}}(\mathbf{v}(r))\mathbf{v}(r)$.

The matrix $\tilde{\mathbf{P}}(\mathbf{v}(r))$ is a sparse, probability transition matrix for joint state $(a, z) \in A_{grid} \times Z$. Its

stationary distribution, g , satisfies

$$\mathbf{g}(r) = \tilde{\mathbf{P}}(\mathbf{v}(r))^\top \mathbf{g}(r).$$

The joint probability of a pair (a_{i_a}, z_{i_z}) given r , $g(a_{i_a}, z_{i_z}; r)$, is stored as the $(I_k \times (i_z - 1) + i_a)$ -th element in the $\mathbf{g}(r)$ vector. The $\tilde{\mathbf{P}}$ matrix, which is used to solve an individual’s optimization problem, is also used to calculate a stationary distribution of individual states. As Rendahl (2022) points out, this property is the same as in the continuous-time method in Achdou et al. (2021). An invariant distribution of $\tilde{\mathbf{P}}(r)$ can be computed instantly by a “dirty trick” described in Achdou et al. (2021) or by repeatedly applying a transition matrix $\tilde{\mathbf{P}}(r)$ from some initial distribution until convergence is achieved.

The aggregate savings given the real interest rate, $A(r)$, can be computed as

$$A(r) = \mathbf{g}(r)^\top \begin{bmatrix} \mathbf{a} \\ \vdots \\ \mathbf{a} \end{bmatrix},$$

where $\mathbf{a} = (a_1, a_2, \dots, a_{I_k})^\top$ is a vector of exogenous grid points and the rightmost vector stacks $\mathbf{a}$ for N_z times. In the Huggett (1993) model, aggregate saving must be zero in an equilibrium. To find an equilibrium real interest rate, one needs to set up an outer loop over r that implements, e.g., a bisection method to find a zero of $A(r)$.

Overall, when solving a heterogeneous-agents model, this paper’s method and the continuous-time method differ only in how they solve the individual optimization problem. The outer loop for the real interest rate and computation of a stationary distribution can be set identical, regardless of whether the model is in discrete or continuous time. Given that this paper’s discrete-time method can solve an individual problem as fast as the continuous-time method, it is expected that they perform equally well in computing an equilibrium.

For the continuous-time method, I use Ben Moll’s code.¹⁶ As before, all display and plotting commands are disabled. In his code, the preference discount rate is set to 0.05, the relative risk aversion to 2, the borrowing limit k_{min} to -0.15, and the asset upper-bound k_{max} to 5. The endowment shock is a two-state process with $z_1 = 0.2$, $z_2 = 0.1$, and the state switches with a Poisson arrival rate of 1.2.

I use the same parameter values except for the discrete-time transition probability matrix. This change is necessary. In the continuous-time model, within a small time interval, dt , the probability of state switching is $1.2dt$. If one interprets a model with $dt = 1$ as a discrete-time model, it exceeds 1, implying that it is

¹⁶http://benjaminmoll.com/wp-content/uploads/2020/06/huggett_equilibrium_iterate.m, last access: September 22, 2026.

Table 6: Computation time for the Huggett model: discrete-time PFI, MPFI and continuous-time implicit method

	$I_k = 10,000$			$I_k = 1,000$		
	Discrete PFI	Discrete MPFI	Continuous	Discrete PFI	Discrete MPFI	Continuous
Total time (sec.)	0.7272	0.5007	0.8187	4.4065×10^{-2}	6.4853×10^{-2}	5.7952×10^{-2}
Outer loop steps	10	10	12	11	11	11

inappropriate for probability. I choose the transition probabilities so that the outer loop for the real interest rate converges in the same number of steps for discrete- and continuous-time models for $I_k = 1,000$, to facilitate speed comparison. This happens when I set $\pi_{1,1} = \pi_{2,2} = 0.8$ and $\pi_{1,2} = \pi_{2,1} = 0.2$.

I use the same algorithms as in Moll’s code to compute a stationary distribution given a real interest rate (“dirty trick”) and to compute an equilibrium interest rate (the bisection method).

With 1,000 grid points, the average computation times under PFI and MPFI are similar to that of the continuous-time method, with PFI being slightly faster and MPFI slightly slower than the continuous-time method. Given that the number of outer-loop steps is the same across all methods, these results indicate that the present paper’s method is an efficient and robust tool in computation for heterogeneous-agent general equilibrium models. With 10,000 grid points, however, both MPFI and PFI are faster than the continuous-time method. The difference is partially explained by the fact that, for PFI and MPFI, the bisection method to search for the equilibrium real rate requires fewer iterations in this exercise. Still, even after accounting for the number of outer-loop steps, MPFI is faster than the continuous-time method. It suggests that MPFI is more useful when the grid size is large than when the grid size is relatively small.

However, this does not imply that the continuous-time method is slower than this paper’s method. In the continuous-time method, one can adjust another parameter, Δ , which is the inverse of the discretized time interval dt , to improve speed. The above numbers are obtained under $\Delta = 1,000$, and they can be improved by using larger Δ . For example, with $\Delta = 10,000$, the average time becomes 4.8187×10^{-2} seconds for $I_k = 1,000$ and 0.6993 seconds for $I_k = 10,000$; with $\Delta = 50,000$, the average time becomes 4.4271×10^{-2} seconds for $I_k = 1,000$ and 0.6863 seconds for $I_k = 10,000$.

Overall, the results suggest that the discrete-time method proposed in the present paper is a useful, comparably fast alternative for a continuous-time method. Although I focus on computation of a stationary equilibrium here, the sequence-space Jacobian method (Auclert et al., 2021) can be applied to compute a dynamic response to aggregate shocks.

E Extension: endogenous labor supply

In this section I extend the income fluctuation problem to incorporate another intra-temporal choice: endogenous labor supply. I assume that the utility is separable between consumption and labor supply, so that the Euler equation can be analytically solved for the current consumption, taking the derivative of the next-period value function.¹⁷

Denoting the labor supply by h , the household's problem is now given by

$$\max_{c, h, a' \leq a_{max}} u(c) - q(h) + \beta \hat{V}(a'; \mathbf{v}^e(z))$$

subject to the budget constraint,

$$c + a' \leq (1 + r)a + wz h,$$

and the borrowing constraint,

$$a' \geq a_{min}.$$

The function q denotes the disutility from labor, and I assume that its derivative, q' , has an analytical inverse. As before, I assume $\mathbf{v}^e(z)$ is such that the continuation value function, $\hat{V}(\cdot; \mathbf{v})$, is concave.

E.1 A one-step approach

The first is a one-step approach, which computes the policy functions for consumption, savings, and labor supply simultaneously.

Fix an arbitrary $z \in Z$. From the first-order condition, $q'(h) = wzu'(c)$ holds, implying $h = (q')^{-1}(wzu'(c))$.

Because both w and z are exogenous, the above equation relates the optimal consumption and labor.

Consider when it is optimal to choose $a' \in (a_i, a_{i+1})$ for $i \in \{1, 2, \dots, I_k - 1\}$. For the optimal savings to be in the interval (a_i, a_{i+1}) , the current consumption must satisfy the Euler equation, implying $u'(c) = \lambda_i$ and hence $c = c_i^*$. From the intratemporal first-order condition, the optimal labor supply is pinned down at $h = h_i^* := (q')^{-1}(wzu'(c_i^*))$. Hence, if the beginning-of-period cash-on-hand, $y^{beg} := (1 + r)a$, is in an interval $\mathcal{I}_i := (c_i^* - wz h_i^* + a_i, c_i^* - wz h_i^* + a_{i+1})$, we have

$$(c, h, a') = (c_i^*, h_i^*, y^{beg} + wz h_i^* - c_i^*).$$

On an $\mathcal{I}$ interval, a' is linear in y^{beg} and both c and h are constant.

¹⁷Barillas and Fernandez-Villaverde (2007) extend the standard EGM to incorporate endogenous labor supply with non-separable utility.

Consider when it is optimal to choose $a' = a_i$ for $i \in \{2, 3, \dots, I_k - 1\}$. As in the model with exogenous labor supply, the Euler equation implies $\lambda_i \leq u'(c) \leq \lambda_{i-1}$, and it follows that $c_{i-1}^* \leq c \leq c_i^*$. Hence, if the beginning-of-period cash-on-hand, y^{beg} , is in an interval $\mathcal{J}_i := (c_{i-1}^* - wz h_{i-1}^* + a_i, c_i^* - wz h_i^* + a_i)$, we have

$$(c, h, a') = (c, (q')^{-1}(wzu'(c)), a_i),$$

where c solves a nonlinear equation,

$$c - wz(q')^{-1}(wzu'(c)) = y^{beg} - a_i. \quad (4)$$

On $\mathcal{J}$ intervals, c and h are non-linear in y^{beg} , whereas a' is constant. This property also holds true for the boundary cases, $a' = a_1$ and $a' = a_{I_k}$.

Note that the policy function for savings, a' , remains piecewise linear, implying that the evaluation of the continuation value reduces to a vector multiplication:

$$\hat{V}(g_{a'}(y_i(z); \mathbf{v}^e(z)); \mathbf{v}^e(z)) = \mathbf{p}(y_i(z); \mathbf{v}^e(z))^\top \mathbf{v}^e(z).$$

Therefore, PFI is again equivalent to solving a large yet sparse system of linear equations, which can be executed very quickly.

Is evaluating the policy function for consumption and labor policies on $\mathcal{J}$ intervals costly? Not significantly. Notice that equation (4) is independent of the continuation value function, $\mathbf{v}^e(z)$. Hence, for any combination of an arbitrary grid point for the beginning-of-period cash on hand, y_j^{beg} , and an arbitrary grid point for the next-period savings, a_i , one can pre-compute $c_{i,j}$ that satisfies equation (4). Therefore, for each $i \in \{1, 2, \dots, I_k - 1\}$, one just needs to find a set of j 's such that $c_{i-1}^* \leq c_{i,j} \leq c_i^*$.¹⁸

E.2 A two-step approach

The second approach is to solve an approximate problem in two steps. It is based on the observation that the household's optimization problem can be written as

$$\max_h -q(h) + m(y; \mathbf{v}^e(z))$$

¹⁸For $i = 1$, one needs to find a set of j 's such that $c_{i,j} \leq c_i^*$, and for $i = I_k$, a set of j 's such that $c_{I_k-1}^* \leq c_{i,j}$.

subject to the budget constraint $y \leq (1 + r)a + wzh$. This is because the $\mathcal{P}(y; \mathbf{v}^e(z))$ -problem can be interpreted as the household's problem after she has chosen her labor supply and, as a result, labor earnings.

This reformulated problem can be approximated by

$$\max_h -q(h) + \hat{M}(y; \mathbf{m})$$

subject to the budget constraint, where $\hat{M}(y; \mathbf{m})$ is a piecewise-linear interpolation of a vector $\mathbf{m} = (m(y_1; \mathbf{v}^e(z)), \dots, m(y_{I_y}; \mathbf{v}^e(z)))^\top$, which is the vector of values of $m(\cdot; \mathbf{v}^e(z))$ on a cash-on-hand grid. The cash-on-hand grid can be either exogenous or endogenous. Because the function m is not in general piecewise linear, this problem is only an approximation of the original problem.

Observe that this approximate problem has the same structure as the problem solved in Proposition 1 after a change of variables.¹⁹ Hence, its solution can be analytically obtained and is a piecewise-linear function.

The two-step approach therefore eliminates the need to solve a nonlinear equation, and the policy function can be evaluated with piecewise-linear interpolation. Because the problem it solves is only an approximation to the problem solved by the one-step approach, the resulting policy function is less accurate than that of the one-step approach.

¹⁹This can be seen by changing variables as follows: $d = -wzh$ and $\tilde{u}(d) = -q(-d/(wz))$. Then the objective function is $\tilde{u}(d) + \beta \hat{V}(y; \mathbf{m}/\beta)$ and the constraint is $d + y \leq (1 + r)a$.